

Lab 3: Component Modelling & Architectural Pattern Selection

NAME-ARYAN N

SRN-PES1UG24CS656

Architecture Selection: *"We chose Layered Architecture for the Self-Service Coffee Kiosk System."*

Architectural Choice

We selected the **Layered architectural style** with three layers: a Presentation layer (Touch Screen UI Component), a Business layer (Order Manager, Payment Service and Receipt Printer Components) and a Data layer (Menu Database Component). Each layer talks only to the layer below it through explicit provided/required interfaces: IOrderService, IPaymentService, IReceiptService and IMenuData. The Order Manager is the single coordinator of the ordering workflow.

Two Reasons for the Choice

1. **The kiosk is a small, single-device system with a fixed sequential workflow.** A customer picks one of 3 drinks and one of 2 sizes, pays by card and receives a receipt. This flow maps directly onto layers with the Order Manager in the middle. Microservices would add service discovery, network calls and deployment overhead that a six-item menu on one machine does not justify, and Client-Server assumes a central server with many clients, which a stand-alone kiosk does not need.
2. **Layering isolates the parts most likely to change.** The touch screen, the receipt printer hardware and the card-payment provider are hardware- or vendor-dependent. Because each sits behind an interface, any of them can be replaced (new screen design, different printer driver, another payment provider) without modifying the others, and new drinks or prices only touch the Data layer. Each component can also be tested independently using stubs.

Security Advantage

The layers enforce controlled access paths. The touch screen UI never handles card data and never accesses the database directly: every request goes through the Order Manager, and only the Payment Service processes card details. Only the Order Manager can reach the Data layer, so a compromised or tampered UI cannot read or change menu prices. This confines sensitive card handling to one component, keeps input validation in one place and reduces the scope of payment-security (PCI-DSS) compliance. Receipts and stored orders keep only masked card information.

Performance Benefit

All layers run in one process on the kiosk, so calls between components are in-process function calls with no network hop or serialisation, which keeps the touch interface responsive (unlike microservices, where every interaction adds network latency). Since the menu is tiny and rarely changes, the Order Manager can load it once through IMenuData and cache it, avoiding a database query on every screen tap.

Trade-off Accepted

Layered systems couple layers more tightly and are hard to scale one component at a time. This is acceptable because each kiosk serves one customer at a time; if many kiosks were later managed centrally, a server-side tier could be added on top of this design.